From a Rust program to one step on Bitcoin

How a withdrawal is proved, wrapped and disputed in LN-GAP 2.5

waxwing

October 2026

An explainer for engineers with a basic familiarity with zero-knowledge proofs. Figures measured on the proof of concept (branch `v25-seal`).

Abstract

A withdrawal from the LN-GAP 2.5 sessions layer is a claim that a program, run on some input, produced some output. Bitcoin cannot run programs, so the claim travels through four machines: RISC Zero proves the program's execution with a STARK; RISC Zero and gnark compress that STARK into a 128-byte Groth16 proof; a fixed RISC-V program, BitVMX's Groth16 verifier, checks that proof in about 478 million steps; and if the two parties disagree about that check, they bisect its execution off-chain until a single RISC-V instruction is left, which Bitcoin Script can check. This note walks through each stage: what goes in, what comes out, why the stage exists, where it runs and what it costs. It ends with who does what in a normal withdrawal and in a dispute, and a glossary.

Contents

1 The problem, and the shape of the answer **2**

2 Background: proofs, STARKs and SNARKs **3**
2.1 What a proof gives you 3
2.2 STARKs and SNARKs 3

3 Stage 1: the guest program **4**

4 Stage 2: the STARK proof **4**

5 Stage 3: the shrink-wrap into Groth16 **5**

6 Stage 4: the input to the BitVMX verifier **6**

7 Stage 5: the verifier program **6**

8 Stage 6: bisecting the verifier's execution **7**

9 Where LN-GAP comes in **8**

10 Who does what, where, and how long 9

11 Practical notes from the proof of concept 9

Glossary 9

1. The problem, and the shape of the answer

Alice has a channel with a hub. Her session contract holds bitcoin that she may withdraw if she proves she returned the hub's tokens on a layer 2 (the L2). Normally the hub checks her proof itself and pays her in the channel; nothing touches Bitcoin. But if the hub refuses, the claim must be settled by Bitcoin, and Bitcoin's Script can do only small things: it has no loops, a stack of 1,000 items and practical transaction sizes of tens of kilobytes. It cannot verify a modern proof directly, let alone re-run a program.

The answer combines two ideas.

1. *Succinct proofs.* Instead of the program's whole execution, Alice shows a short proof that the execution happened and produced a given output. Checking the proof is far cheaper than re-running the program, and the proof is the same size whatever the program.
2. *Optimistic bisection.* Even checking the proof is too much for Script (hundreds of millions of steps). But checking it is itself a deterministic program. If Alice and the hub disagree about its result, they can binary-search its execution for the first step on which they disagree. Bitcoin then only has to check that one step.

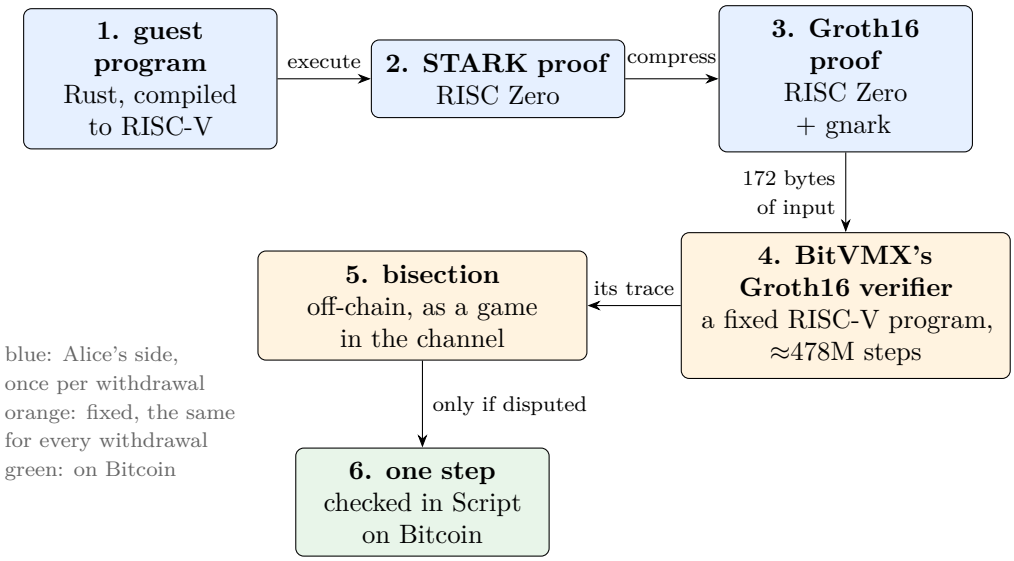


Figure 1: The pipeline. Stages 1–3 make a proof; stage 4 is the one program anyone ever disputes; stages 5–6 happen only if the hub disagrees with Alice about the result of stage 4.

There are **two different RISC-V programs** in this picture, run by **two different machines**. Our statement (the guest) runs only in RISC Zero's zkVM, on Alice's computer. BitVMX never sees it. BitVMX's emulator runs only the Groth16 verifier, and it is the verifier's execution that is bisected. The verifier is the same program for every withdrawal and every statement.

2. Background: proofs, STARKs and SNARKs

2.1. What a proof gives you

A proof system lets a *prover* convince a *verifier* that a *statement* is true. For us the statement is always of one form:

"program P, run on some input w, terminates and outputs J."

The output J is public (it is called the *journal*); the input w is the prover's *witness* and may stay private. A proof system for such statements offers:

- *Soundness:* if the statement is false, no prover can make the verifier accept, except with negligible probability.
- *Succinctness:* the proof is small and fast to check, much smaller and faster than re-running P.
- *Zero knowledge* (optional): the proof reveals nothing about w beyond what J says. This is what makes private payments possible; for the withdrawal itself we only need the first two.

Under the hood, a proof system turns the execution into a large set of polynomial equations over a finite field (an *arithmetisation*) and proves that the equations hold without sending them all. Two families of systems matter here, and they make opposite trade-offs.

2.2. STARKs and SNARKs

	STARK (as in RISC Zero)	Groth16 (a pairing-based SNARK)
statement size	any program, any length (split into segments)	one fixed circuit
setup	transparent: no secrets	a trusted setup per circuit (a ceremony)
cryptography	hashes only (here Poseidon2 over the BabyBear field)	elliptic-curve pairings (here the BN254 curve)
proof size	hundreds of kilobytes	3 curve points: 256 bytes, 128 compressed
verifier cost	many hash computations	3 pairings and a few curve operations
prover cost	fast	slow, needs a lot of memory

The pipeline uses each for what it is good at. RISC Zero's STARKs prove an arbitrary program cheaply. A Groth16 proof is what you want to hand to a constrained verifier: small, with a fixed verification procedure. So the STARK is proved first, and then a fixed Groth16 circuit proves the single statement "this STARK verifies". Because that circuit is the same for every program, one trusted setup serves everyone; RISC Zero ran a public ceremony for it.

Fields. Each system computes in its own finite field. RISC Zero's STARK works modulo the 31-bit prime $p = 15 \cdot 2^{27} + 1$ ("BabyBear"), which suits fast hardware arithmetic. Groth16 over BN254 works modulo a 254-bit prime. Checking a BabyBear hash inside a BN254 circuit is very expensive, and this mismatch is the reason for an extra step in Section 5.

3. Stage 1: the guest program

The statement is an ordinary Rust program, called the *guest*, compiled for RISC Zero's zkVM: a 32-bit RISC-V machine (the RV32IM instruction set: integer arithmetic, multiplication, division). It reads private input from the host and writes public output to the journal. The placeholder guest of the proof of concept:

```
use risc0_zkvm::guest::env;

fn main() {
    let (b, c): (u32, u32) = env::read();   // private input
    let mut journal = [0u8; 8];
    journal[..4].copy_from_slice(&b.to_le_bytes());
    journal[4..].copy_from_slice(&c.to_le_bytes());
    env::commit_slice(&journal);            // public output
}
```

It checks nothing; the real withdrawal guest will check the L2 sequencer's signature on a state root and the path of Alice's return note in that state (it reads them as private input) and commit b, the amount, and c, the session's identifier.

The image id. The compiled program is identified by its *image id*, a 32-byte hash of its memory image. A proof is always about a particular image id: "the program with image id X output J". Anyone checking a withdrawal must therefore check that X is the withdrawal guest's image id; otherwise Alice could prove the output of a different program that prints whatever she likes. The image id changes with any change to the code *or to the toolchain*, so both parties must build from the same commit with the same tools.

Where it runs. Anywhere. The guest is compiled with RISC Zero's Rust toolchain (installed by its `rzup` tool) on the Mac.

4. Stage 2: the STARK proof

RISC Zero first *executes* the guest, recording a trace of every cycle: the registers, the memory accessed, the instruction. It splits long executions into *segments* (about a million cycles each) and proves each segment with its RISC-V circuit, a STARK. Then it proves *the verification of those proofs*, in a smaller circuit made for verifying STARKs (the *recursion* circuit), which lets it fold any number of segment proofs into one:

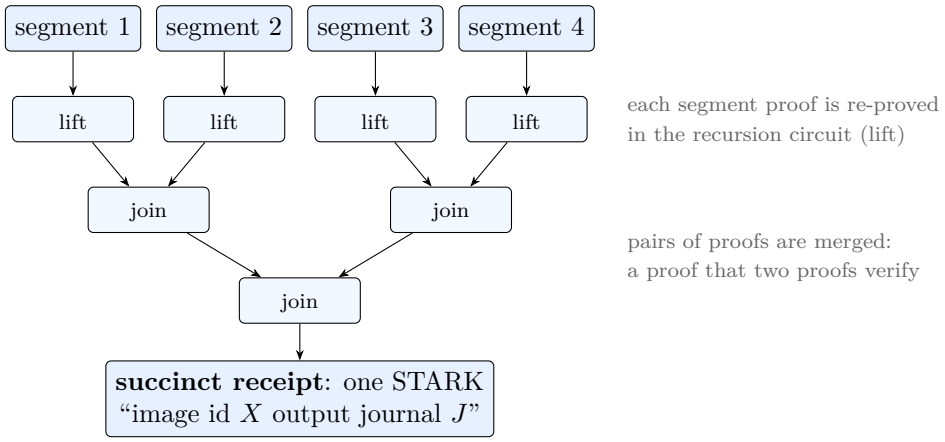


Figure 2: Recursion. A long execution is split into segments; each segment's STARK is lifted into the recursion circuit, and pairs are joined until one proof remains. Our placeholder guest is a single segment (3,883 cycles); the Groth16 verifier, if it were the guest, would be hundreds.

The result is the *succinct receipt*: one STARK, of fixed size (about 220 KB), whatever the length of the execution, saying that the program with image id X ran and output J. Every recursion step is itself a program for the recursion circuit, identified by a *control id*; the set of allowed recursion programs is committed to by a Merkle root, the *control root*, which a verifier must know. The control root is fixed per RISC Zero release, which is why the version matters later (Section 7).

Where it runs, and how long. Anywhere: the proof of concept proves on the Mac (about 7 s for the placeholder guest, using RISC Zero's prebuilt prover `r0vm`) and on an x86 Linux laptop (21 s, 12 cores). The proving time grows with the guest's cycles; a real withdrawal guest (a signature check and a Merkle path) stays small.

5. Stage 3: the shrink-wrap into Groth16

The succinct receipt is a STARK of 220 KB whose verification needs many BabyBear hash computations. To get a small proof with a fixed verifier, RISC Zero wraps it in Groth16, in two steps.

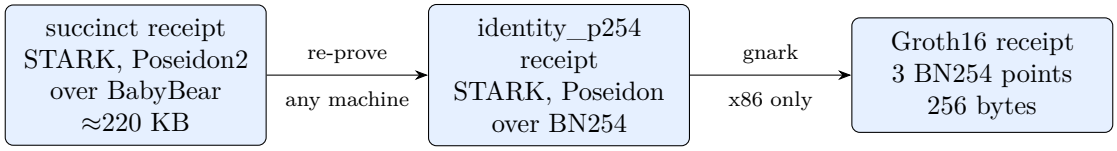


Figure 3: The shrink-wrap. The first step changes the hash the STARK is built with, the second proves “this STARK verifies” in a fixed Groth16 circuit.

Why the proof is re-proved first (identity_p254). The Groth16 circuit is a circuit over BN254's 254-bit field. Inside it, every BabyBear operation of the STARK's verification would have to be emulated with non-native arithmetic: far too large. So RISC Zero first runs one more

recursion step, *identity_p254*, whose job is only to re-prove the succinct receipt with a STARK whose commitments use a Poseidon hash over BN254's own field. Verifying *that* STARK inside the BN254 circuit is affordable. Each step takes the previous step's output as its input, and each layer exists because the next one could not take the original directly: arbitrary execution, then one succinct STARK, then a BN254-friendly STARK, then Groth16.

The Groth16 step. A fixed circuit verifies an identity_p254 STARK and outputs a Groth16 proof: three points on the BN254 curve, A and C on the group G_1 (64 bytes each, two 32-byte coordinates) and B on G_2 (128 bytes), 256 bytes in all. Its *public inputs*, the values a verifier must supply, are:

- the control root (Section 4), as two 128-bit halves: which recursion programs were allowed;
- the *claim digest*, as two halves: a hash of the image id, the journal and the exit status;
- the BN254 control id: which identity_p254 program was used.

The verifier recomputes the claim digest itself from the image id and the journal it is given, so the proof is bound to exactly that program and that output.

Where it runs, and how long. RISC Zero runs the Groth16 prover (gnark, with its proving key) in a Docker image built for x86 only. This is the one step that needs an x86 machine. On the 12-core Linux laptop, the whole wrap took 102 s; it needs several gigabytes of memory (it was killed at under 2 GB on the Mac's small Docker VM).

6. Stage 4: the input to the BitVMX verifier

BitVMX's Groth16 verifier takes its input as a flat byte string. Points are stored compressed: only the x coordinate and a sign bit, since y can be recomputed from the curve equation. That makes A and C 32 bytes and B 64:

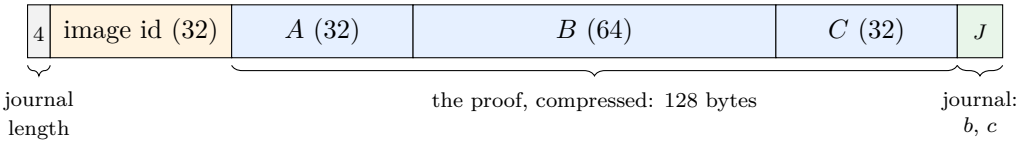


Figure 4: The verifier's 172-byte input for our guest. The journal length is counted in 4-byte words (here 2). The sample proof used before had a 4-byte journal and so 168 bytes.

Because the journal is part of the input, b and c sit at fixed byte offsets, and so in fixed 4-byte input words. That is what lets the Bitcoin contract read them later: Alice signs every input word with a one-time key when the dispute starts, and the contract's payout scripts read the bits of b from her signature on that word.

7. Stage 5: the verifier program

The verifier is a C program (FairgateLabs' `bitvmx-zk-verifier`) compiled to RISC-V for BitVMX's emulator. It is fixed: the same binary checks every withdrawal. Its constants are

baked in: the Groth16 verifying key of RISC Zero's circuit, the control root and the BN254 control id. Given the 172 bytes, it

1. decompresses the three points;
2. recomputes the claim digest from the image id and the journal;
3. computes the linear combination of verifying-key points weighted by the public inputs;
4. checks the pairing equation $e(A, B) = e(\alpha, \beta) \cdot e(L, \gamma) \cdot e(C, \delta)$, where $\alpha, \beta, \gamma, \delta$ come from the verifying key and L is the combination of step 3;
5. halts with exit code 0 if it holds, otherwise non-zero.

Pairings on a 32-bit machine without big-number instructions are slow: the run on our receipt took **477,641,148 steps** (57 s in the emulator on the Mac) and halted with exit code 0. With the journal's b changed from 9 to 10 it halted with exit code 1: the proof is bound to the journal.

Why versions matter. The baked constants belong to one RISC Zero release family. The proof of concept found that the shipped verifier carries RISC Zero 3.0's control root (`a54dc85a…`), so receipts made with RISC Zero 3.0.x verify with it unchanged; an earlier sample proof, made with older parameters, needed the constants patched in the binary. The proving tools are therefore pinned to RISC Zero 3.0.6.

8. Stage 6: bisecting the verifier's execution

Suppose Alice claims "the verifier, run on these 172 bytes, halts with exit code 0" and the hub disagrees. Both can run the verifier locally (57 s), and both are bound to the same input, since Alice has signed it. Instead of asking Bitcoin to run 478 million steps, they search for the first step on which they disagree.

State hashes. BitVMX summarises the machine after step i by a hash h_i, chained so that h_i depends on h_{i-1} and on what step i did (the instruction, the values it read and wrote, the next program counter). Agreeing on h_i means agreeing on the whole execution up to step i.

The search. Alice (the prover) and the hub (the verifier) both know h_0 (the program's start) and disagree about the end. In each round Alice publishes the hash at the midpoint of the current interval; the hub compares it with its own and picks the half in which they disagree. After $\log_2$ of the number of steps rounds, the interval is a single step $k \to k + 1$: they agree on h_k and disagree on h_{k+1}.

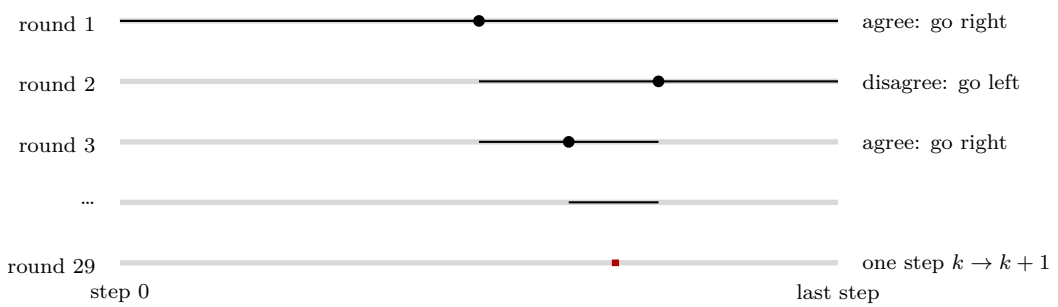


Figure 5: Binary search over the verifier's execution. In each round the hub compares Alice's midpoint hash with its own and keeps the half in which they disagree; at the end they agree on h_k but not on h_{k+1}. For the Groth16 verifier, 29 rounds narrow 2^{29} steps to one.

The last step. Now Bitcoin has to judge one RISC-V instruction: given the state committed by h_k, does executing step $k+1$ produce h_{k+1}? Alice reveals the step's details (the instruction, the registers and memory it read, what it wrote) and a script checks that they hash to the agreed h_k, that the instruction computes what it claims, and that the result hashes to her h_{k+1}. If the hub suspects a value she claims to have read, a second, shorter search (the *read challenge*) finds the step that last wrote that address. In the proof of concept the final step of the verifier, its halting system call, is proved in a 58.5 kvB transaction; the disputed step is a single instruction, so the cost does not depend on the program.

Honesty is enough. An honest Alice can always answer, because her hashes come from the true execution; an honest hub can always find a disagreement with a false claim, because the true execution ends differently. Whoever is wrong runs out of moves.

9. Where LN-GAP comes in

BitVMX plays the search on Bitcoin: every round is a transaction with a timeout. LN-GAP 2.5 plays it instead as a game inside the channel. Each move (a midpoint, a choice) is a signed message between Alice and the hub, so a withdrawal dispute between honest parties costs nothing on chain until its last step. What the game needs from outside is a record of *when* each move was made, so that a party who stops answering can be told apart from one whose answer was suppressed. That record is kept by a venue of bonded members, who put Merkle roots of the moves they received on Bitcoin in their periodic seal closings; a party claiming its opponent stalled is answered by a rebuttal that proves the move was dated in time. Only when someone misbehaves (stalls, makes a false claim, or a member fails) does the game continue on chain, one move per transaction, at the cost of the bond of whoever forced it. LN-GAP 2.5 itself is not published; its successor, Lichen (`https://github.com/AdamISZ/lichen`), drops the venue and plays the search on Bitcoin, as BitVMX does. For the proving pipeline, what matters is that the program being searched is always the Groth16 verifier of Section 7.

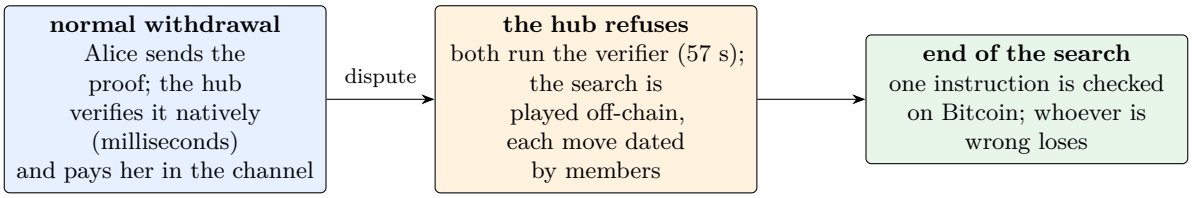


Figure 6: Most withdrawals stop at the first box. The search is played only when the hub disputes, and reaches Bitcoin only at its end.

10. Who does what, where, and how long

step	who	where	measured
compile the guest	both (to agree on the image id)	any machine	seconds
prove (STARK)	Alice	any machine	7 s (Mac), 21 s (x86, 12 cores)
wrap (identity_p254 + Groth16)	Alice	x86 for the Groth16 step	102 s (x86, 12 cores); several GB
verify natively	the hub	any machine	milliseconds
run the BitVMX verifier	both, only in a dispute	any machine	478M steps, 57 s
the search	both, only in a dispute	off-chain, in the channel	29 rounds (59 moves)
the last step	Alice proves it, or the hub's challenge stands	Bitcoin	58.5 kvB

11. Practical notes from the proof of concept

- *Pin the RISC Zero version* to the release whose constants the verifier binary carries (3.0.x for the shipped binary). The image id depends on the toolchain too.
- *The Groth16 step needs x86 and Docker.* On Apple silicon, RISC Zero's in-process prover also needs Xcode's Metal compiler; using the prebuilt `r0vm` through RISC Zero's client API avoids it.
- *Snap-installed Docker* cannot see `/tmp`, where RISC Zero hands the Groth16 step its files; set `RISC0_WORK_DIR` to a directory under your home.
- *Memory:* the Groth16 step was killed on a 2 GB VM and ran on a 38 GB laptop; budget 16 GB.
- *Encoding:* the verifier wants compressed points and the journal length in words; FairgateLabs' `proof-as-input` tool produces the layout of Figure 4.

Glossary

BabyBear
the 31-bit prime field $15 \cdot 2^{27} + 1$ RISC Zero's STARKs compute in.

BN254
a pairing-friendly elliptic curve (also called alt_bn128); Groth16 proofs here live on it.

BitVMX
a protocol and RISC-V emulator for disputing a program's execution on Bitcoin by bisection, by FairgateLabs.

claim digest
a hash of the image id, the journal and the exit status: what a RISC Zero proof is ultimately about.

control id, control root
identifiers of RISC Zero's recursion programs, and the Merkle root of the allowed ones; fixed per release.

gnark
a Go library for SNARKs; RISC Zero uses it for the Groth16 step.

Groth16
a pairing-based SNARK with constant-size proofs (three curve points) and a per-circuit trusted setup.

guest
the program whose execution is proved, run in RISC Zero's zkVM.

identity_p254
the recursion step that re-proves a STARK with a BN254-friendly hash, so that Groth16 can verify it.

image id
the hash of a guest's compiled memory image.

journal
the guest's public output.

receipt
RISC Zero's proof object (a journal plus a seal): segment, succinct, or Groth16.

recursion
proving the verification of proofs, to fold many into one.

segment
a slice of a long execution, proved separately.

shrink-wrap
the conversion of a succinct receipt into a Groth16 receipt.

STARK
a hash-based proof system with a transparent setup and larger proofs.

witness
the private input that makes a statement true.

zkVM
: a virtual machine whose executions can be proved.